

Assignment #1 Parking Fee Calculator

Due Date: September 30, 2026 (Wed), 23:59 (KST)

Contact: Sumin Leem (leemsmn@postech.ac.kr)

GOAL

Implement two programs: one calculates parking fees from parking durations; the other finds the maximum parking time for a given budget. Use variable declarations, integer arithmetic, conditionals, and loops, and explain how you verified your implementation in the report.



Figure 1. Parking fee calculator. See the Parking Fee table below for the fee rules.

Parking Fee	
Item	Rule
Free period	0 to 30 minutes, inclusive
Additional fee	Round the time beyond 30 minutes up to 10-minute units; charge 500 won per unit.
Maximum fee	5,000 won per car

For example, parking for 41 minutes leaves 11 chargeable minutes after the free period. These count as two units, so the fee is 1,000 won.

The maximum parking time is 24 hours. Parking time is given as an integer from 0 to 1440 minutes, inclusive. Calculate the fee for each car independently.

PROBLEMS

1. Parking Fee Calculation (45 points)

Calculate the parking fee for each of N cars, and print the total fee and the number of free cars. A car with a fee of 0 won is counted as a free car.

[Input]

Enter the number of cars N ($1 \leq N \leq 20$). Then follow the prompts to enter each car's parking time in minutes, one at a time. Press Enter after each number.

- Input in progress:

```
===Parking Fee Calculator===
Number of cars: 3
Car 1 parking time (minutes): 43
Car 1 fee: 1000 won
Car 2 parking time (minutes):
```

The user has entered 3 cars and 43 minutes for the first car, and the program has printed a fee of 1,000 won. It is now waiting for the second car's parking time.

[Output]

Match the output format in the example below, including the input prompts. No points will be deducted for trailing spaces, extra spaces, or differences in alignment.

- Example run: The user enters 5 cars, followed by the parking times in order.

Car	1	2	3	4	5
Parking time (min)	0	30	31	41	131

The example below shows both program output and user input.

```
===Parking Fee Calculator===
Number of cars: 5
Car 1 parking time (minutes): 0
Car 1 fee: 0 won
Car 2 parking time (minutes): 30
Car 2 fee: 0 won
Car 3 parking time (minutes): 31
Car 3 fee: 500 won
Car 4 parking time (minutes): 41
Car 4 fee: 1000 won
Car 5 parking time (minutes): 131
Car 5 fee: 5000 won
Total fee: 6500 won, Free cars: 2
```

[Implementation Requirements]

Use a loop to process N cars. You may choose whether to use user-defined functions. Write a complete program with the required headers and main, and save it as `problem1.c`.

2. Maximum Parking Time (45 points)

Read a budget in won and print the longest affordable parking time as an integer in minutes. Apply the parking fee rules above and the 24-hour maximum parking time.

- budget is a nonnegative integer representable by int. It may not be a multiple of 500.
 - Parking is allowed when the fee equals the budget.
 - You may choose your solution method and whether to use user-defined functions.
 - Execution time optimization is not required.
- Example run: The user enters a budget of 750 won. Match the example's output format, including input prompts. The same rules for spacing and alignment as in Problem 1 apply.

```
===Maximum Parking Time===  
Budget (won): 750  
Maximum parking time: 40 minutes
```

[Implementation Requirements]

Write a complete program with the required headers and main, and save it as `problem2.c`.

GUIDELINES

- Do not use AI. **Write the code yourself.** POSTECH's academic misconduct rules apply.
- Follow each problem's scope and requirements. Extra features earn no extra credit.
- Follow the format of the examples in the problems to support fair grading.
- Write in C and use `scanf` and `printf` for input and output. Do not use arrays, recursion, global variables, or `goto`.
- Declare all variables inside the function that uses them. Descriptive variable names and consistent indentation are recommended.
- Assume that all input satisfies the specified ranges and formats.

- **Grading uses Visual Studio 2022 on Windows.** See the attached setup document. You may develop elsewhere, but submitted sources must build and run here without installing additional libraries.
- To use `scanf` in Visual Studio, place the following line at the very top of each source file.
`#define _CRT_SECURE_NO_WARNINGS`

SUBMISSION

Submission location: Assignment #1 submission page on PLMS

Submission format: ZIP problem1.c, problem2.c, and report.pdf as **assn1_StudentID_Name.zip**.

REPORT

Submit one concise report.pdf covering the following main points. **Do not include program source code in the report.**

- Basic information:
 - Student ID, department, name, and HEMOS ID
- Development environment:
 - Operating System, IDE, and version
- Implementation design:
 - Explain each problem's requirements and your design, including calculations, conditionals, and loops. Relate them to your code; variable names and formulas are allowed.
- Execution results and verification:
 - For each problem, list inputs and expected results. Compare these with screenshots of the actual results and briefly explain the comparison.
- If relevant, describe implementation problems and how you resolved them.
- Cite any references you used.
- Include any additional information needed to understand your program (optional).

PROGRAM

- Submit Problem 1 as `problem1.c` and Problem 2 as `problem2.c`. Include the required headers and `main` in each file so that **each program runs independently**.
- If a program fails to compile or complete execution, the Program score for that problem is 0. The other problem and the report are graded separately.

SCORING CRITERIA

- The total is 100 points: Program 90 and Report 10.
 - Program — Problem 1 (45 points): Calculate parking fees and process multiple vehicles as required.
 - Program — Problem 2 (45 points): Calculate the maximum parking time for a given budget and handle boundary cases.
 - Report (10 points): Follow the required format, include the required contents, and explain your implementation design.
- Alternative solutions that meet the requirements are accepted. Partial credit is awarded for correctly implemented or explained parts.
- Compilation or execution failure results in 0 for that problem's Program score. The other problem and the report are graded separately.
- Missing code or a missing report receives 0 for that item.

DEDUCTION

- Missing the deadline incurs a deduction of 10% of the assignment maximum (10 points).
- If you submit more than once, only the final submission is graded. Lateness is determined by its completed submission time on PLMS.
- For every additional 24 hours after the deadline, another 10% of the maximum (10 points) is deducted.
- Submissions made 72 hours (3 days) or more after the deadline receive 0 points.
 - 8 hours late: 10% (10 points) deducted
 - 25 hours late: 20% (20 points) deducted
 - 50 hours late: 30% (30 points) deducted
 - 72 hours or more late: 0 points

Deductions are subtracted from the raw score, with a minimum final score of 0. For questions, use email or the course Q&A board on PLMS.